



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
DEEP LEARNING PROJECT (September 16th 2025)

Nanosocrates: Foundational Models Building a Very Small Semantic Language Model

Professor
Prof. Eng. Vito Walter Anelli

Student
Michele Gentile

Academic Year 2024/2025

Contents

1	Introduction	1
1.1	Objective of the project	1
1.2	Database creation	2
1.3	Tokenizer	4
1.3.1	BPE Tokenizer	4
1.3.2	Decoder and Trainer	5
2	Architecture	6
2.1	Transformers	6
2.2	NanoSocrates architecture	7
2.2.1	Encoder	7
	Multi-Head Attention with relative position encoding	8
	Multi-Head Latent attention with Decoupled RoPE	9
	Interleaved Attention Layers	11
	Feed Forward Network	12
2.2.2	Decoder	12
2.2.3	Projection Layer and Softmax	12
3	Training of the model	13
3.1	Training a transformer	13
3.2	Training of NanoSocrates	13
3.2.1	Training setup	14
3.2.2	Input sequences	15
3.2.3	Sanity check on the model	16
3.2.4	Setting of the parameters	17
4	Evaluation of the model	18
4.1	Learning Curves	18
4.2	Text2RDF	19
4.3	RDF2Text	20
4.4	RDF Completion 1 - Spanned Masking	21
4.5	RDF Completion 2	21
5	Conclusions and future Developments	23

Chapter 1

Introduction

Foundational models are very large machine-learning models trained on broad, diverse datasets with the explicit goal of serving as a reusable base for many downstream tasks. Rather than building a separate, task-specific model for every problem, a single foundational model is exposed during pretraining to vast amounts of heterogeneous data—text, images, code, audio or combinations thereof—so that it learns wide-ranging statistical patterns, linguistic regularities, factual knowledge and modality-specific representations. Once pretrained, the model can be adapted to concrete applications through techniques such as fine-tuning on labeled examples, prompt engineering or in-context learning at inference time, and lightweight adapter methods that change only a small fraction of parameters; these approaches let practitioners trade off performance, data needs, and computational cost depending on the task. Because the model encodes broad priors, it often generalizes to tasks it was not explicitly trained for and can demonstrate emergent capabilities, but this generality also brings challenges: models may hallucinate, reflect biases present in training data, require substantial compute to train and serve, and need careful evaluation and governance when deployed in real-world systems.

1.1 Objective of the project

The primary objective of this project is to create a small foundational model from scratch, named **NanoSocrates**, based on a T5-like Encoder-Decoder Transformer architecture. The model will be designed to understand and translate between unstructured natural language and structured data in the form of Resource Description Framework (RDF) triples. RDF is a World Wide Web standard for representing and sharing structured metadata, utilizing a model based on subject-predicate-object triples that form graphs. This project focuses on the movie domain and encompasses four main tasks:

- **RDF2Text**: Converting RDF triples into natural language. The model must comprehend the semantic meaning of the triples and articulate the concept fluently in natural language.
- **Text2RDF**: Converting natural language text into RDF triples. This is the inverse

of the previous task, requiring the model to understand the meaning of the text and structure the information into RDF triples.

- **RDF Completion 1:** In this task, a part of an RDF triple is "masked" with a MASK token. The objective is for the model to predict the missing part of the triple (i.e., the subject, predicate, or object and its corresponding resource).
- **RDF Completion 2:** Given a variable number of input triples, the model must predict the subsequent triples that logically follow.

Beyond the model itself, a key objective is the creation of a custom dataset for training. This dataset will contain labeled examples for all four tasks, each identified by a special token: <Text2RDF>, <RDF2Text>, <CONTINUERDF>, and <MASK>. The model must be able to recognize the task type from the tag present in each example. Following the dataset's creation, the next step is to train a tokenizer from scratch on the corpus derived from the dataset examples, ensuring the inclusion of these special tokens.

1.2 Database creation

The database was built by orchestrating a reproducible, fault-tolerant pipeline that harvests film entities from DBpedia and enriches them with English textual summaries and curated RDF facts.

First, to cope with flaky public endpoints, all HTTP traffic is funneled through a session configured with bounded retries, exponential backoff, and per-request timeouts, ensuring robustness without stalling indefinitely. Film resources are selected via a SPARQL query that orders candidates by a seeded MD5 hash of their URIs; this yields a pseudo-random yet deterministic ordering, enabling exact reproducibility across runs and stable pagination via LIMIT/OFFSET.

For each film, we resolve its canonical English Wikipedia page (via the FOAF primary-topic link, filtered to the en domain) and fetch the short summary through Wikipedia's REST "summary" endpoint; we then retain only the first paragraph to minimize noise and keep inputs concise. On the structured side, we query DBpedia for a whitelist of predicates, accepting either URI objects or literals, while filtering literals by language (English-only) and conservatively truncating overly long values to prevent pathological outliers from dominating samples. The limit was set to 300 characters.

We deduplicate triples and impose a deterministic order using compact CURIE-like names (*qname*) for both predicates and URI objects; this design choice makes the output stable for versioning, human-readable, and suitable for diff-based auditing. We also add the triples token delimiters <SOT> and <EOT>, to mark respectively the start and the end of the triple. Each example is then packaged for multiple learning tasks by serializing the triples into a linear format and forming paired inputs/targets for Text2RDF and RDF2Text, together with the two completion tasks driven by a locally seeded RNG. For

RDF Completion 1, we mask pseudo-randomly one of the possible roles in the triples, while for RDF Completion 2 we choose a random k . We will consider as input from the first to the $(k-1)$ -th triple, while as target from the k -th triple to the last. To create an example for RDF Completion 2, the film must have at least three triples.

To be a “good citizen” against rate limits, the pipeline supports optional inter-sample sleep; network exceptions are logged and skipped to keep throughput high without aborting the entire run. Finally, records are persisted in JSON Lines (one example per line), chosen for its streamability, incremental appendability, and compatibility with large-scale pre-processing, while directory creation and literal escaping utilities ensure filesystem safety and serialization correctness.

For the database construction, 350 films were selected, yielding a total of 1,206 examples, indeed from each film, we extract four samples, one corresponding to each task. It is possible to configure the maximum number of films, the SPARQL query page size, the output directory for database creation, the sleep time between consecutive requests, the seed for pseudo-randomness, the maximum number of triples, and the selected predicates, respectively, via the arguments `--limit`, `--page-size`, `--outdir`, `--sleep`, `--max-triples`, and `--predicates`. The Python script related to this section is `database_creator.py`. Below, we illustrate one example per task drawn from the resulting dataset.

RDF2Text

```
{
  "input": "<SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:director<OBJ>dbr:James_W._Horne<EOT><SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:starring<OBJ>dbr:Grace_Darmond<EOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:starring<OBJ>dbr:Warner_Baxter<EOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:writer<OBJ>dbr:Wyndham_Gittens<EOT><RDF2Text>",
  "target": "Alimony is a 1924 American silent drama film directed by James W . Horne and starring Grace Darmond, Warner Baxter, and Ruby Miller. In the United Kingdom it was released under the title When the Crash Came ."
}
```

RDF Completion 1

```
{
  "input": "<SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:director<MASK><EOT>",
  "target": "<OBJ>dbr:James_W._Horne"
}
```

Text2RDF

```
{
  "input": "Alimony is a 1924 American silent drama film directed by James W.
    Horne and starring Grace Darmond, Warner Baxter, and Ruby Miller. In
    the United Kingdom it was released under the title When the Crash Came
    .<Text2RDF>",
  "target": "<SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:director<OBJ>dbr:
    James_W._Horne<EOT><SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:starring
    <OBJ>dbr:Grace_Darmond<EOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:
    starring<OBJ>dbr:Warner_Baxter<EOT><SUBJ>dbr:Alimony_(1924_film)<PRED>
    dbo:writer<OBJ>dbr:Wyndham_Gittens<EOT>"
}
```

RDF Completion 2

```
{
  "input": "<SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:director<OBJ>dbr:
    James_W._Horne<EOT><SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:starring
    <OBJ>dbr:Grace_Darmond<EOT><SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:
    starring<OBJ>dbr:Ruby_Miller_(actress)<EOT><SOT><SUBJ>dbr:Alimony_(1924
    _film)<PRED>dbo:starring<OBJ>dbr:Warner_Baxter<EOT><SOT><SUBJ>dbr:
    Alimony_(1924_film)<PRED>dbo:writer<OBJ>dbr:E._Magnus_Ingleton<EOT><
    CONTINUERDF>",
  "target": "<SOT><SUBJ>dbr:Alimony_(1924_film)<PRED>dbo:writer<OBJ>dbr:
    Wyndham_Gittens<EOT>"
}
```

1.3 Tokenizer

From the database obtained by `database_creator.py`, a byte-level BPE tokenizer is trained to ensure full Unicode coverage, robustness to rare symbols, and stable segmentation across heterogeneous inputs.

1.3.1 BPE Tokenizer

A **byte-level BPE tokenizer** segments raw text into subword units that are learned from data to balance coverage and efficiency. Training starts from a base alphabet (here, bytes, so every Unicode symbol is representable), then repeatedly merges the most frequent adjacent token pairs in the corpus, creating new subwords that better match common morphemes, names, and multi-character patterns. This greedy merge process continues until a predefined capacity is reached, yielding a vocabulary whose items range from

single bytes to frequent multi-byte chunks. The *vocabulary size* sets the upper bound on how many distinct tokens the model may use: a larger vocabulary reduces average sequence length (fewer tokens per sentence) and captures richer morphology, but increases embedding/softmax memory; a smaller vocabulary improves parameter efficiency and generalization, but produces longer sequences and may fragment common words into many pieces, which can hurt downstream fluency and latency. The *minimum frequency* acts as a pruning threshold during training: candidate tokens whose corpus count falls below this value are not kept (or not considered for merges), preventing the vocabulary from being diluted by rare, non-generalizable fragments and keeping merges focused on statistically robust patterns. In byte-level BPE specifically, out-of-vocabulary errors do not occur, indeed unseen strings can always be expressed as sequences of known bytes, so the main design trade-off becomes how aggressively to compress frequent patterns into larger subwords (via vocabulary size and minimum frequency) versus keeping sequences more granular. Appropriate settings therefore depend on corpus scale and domain: higher vocabulary sizes and lower thresholds favor compact sequences for repetitive technical domains; more conservative settings prioritize robustness across heterogeneous inputs. In this case we choose a vocabulary size of **16000** tokens to keep model's embedding layer small and memory-efficient. The minimum frequency is set to **2**, so that each token that appears only once in the corpus is not merged, that means a "cleaner" vocabulary.

1.3.2 Decoder and Trainer

A byte-level pre-tokenizer/decoder is adopted so that every character can be represented without out-of-vocabulary failures, an explicit choice for noisy web text and RDF serializations. The trainer reserves core special tokens `[UNK]`, `[PAD]`, `[BOS]`, `[EOS]` and registers task-specific markers (e.g., the tags used to disambiguate Text2RDF/RDF2Text and completion tasks and the start/end of triple markers), guaranteeing that such control tokens belong to the learned vocabulary and are never split by BPE merges. Post-processing is intentionally neutral—single sequence as `$A`, pair as `$A $B`—so that `[BOS]` and `[EOS]` are injected explicitly by the model code, preserving full control over task prompts and sequence formatting. For portability, the tokenizer is exported both as a raw *tokenizers* JSON and as a Hugging Face `PreTrainedTokenizerFast` package (with the special tokens and additional task markers registered), enabling seamless loading via `from_pretrained`, reproducible preprocessing across pipelines, and straightforward sharing alongside the dataset and checkpoints.

Chapter 2

Architecture

Before discussing the **NanoSocrates**' architecture implemented to realize the tasks described in the previous section, we introduce a theoretical foundation for Transformers.

2.1 Transformers

Transformers were introduced by Vaswani *et al.* in the paper *Attention Is All You Need* (2017), proposing a sequence model that relies exclusively on attention mechanisms, dispensing with recurrence and convolution. The key motivation is to enable *parallel* computation over sequence positions and to obtain long-range dependency modeling without the vanishing-gradient and path-length limitations of RNNs/CNNs.

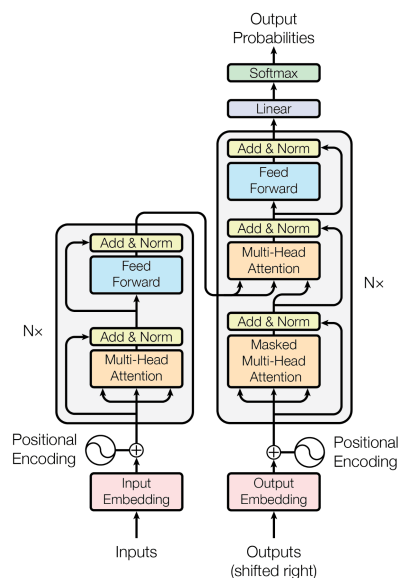


Figure 2.1: Transformer architecture presented in the paper "Attention is all you need" (2017)

The canonical Transformer is a *stacked encoder-decoder* architecture. The **encoder** maps an input token sequence to a sequence of contextual representations, while the

decoder autoregressively generates output tokens using self-attention over the generated prefix and *cross-attention* over the encoder outputs. Each encoder (and decoder) layer is composed of a multi-head (self-)attention sublayer, a position-wise feed-forward network (FFN) and residual connections with layer normalization around each sublayer. The decoder inserts an additional cross-attention sublayer between its self-attention and FFN. Tokens are embedded into vectors in $R^{d_{\text{model}}}$ and scaled by $\sqrt{d_{\text{model}}}$ before entering the stack, where d_{model} is the latent vector dimension. Because attention alone is permutation-invariant, **positional encodings** are added to token embeddings to inject order.

2.2 NanoSocrates architecture

The model architecture used in this project is inspired by models such as T5 (Text-to-Text Transfer Transformer). T5 is a language model developed by Google AI that unifies natural language processing (NLP) tasks into a single text-to-text format and is always composed of a stack of encoders and decoders. NanoSocrates shares with T5 the layered structure and the relative position encoding, which is used to give the model a notion of the relative positions of tokens within a sentence. In NanoSocrates, we have **4** layers of Encoders / Decoders. In the next section, we will see piece by piece the architecture of NanoSocrates, starting from the Encoder stack.

2.2.1 Encoder

The Encoder structure of NanoSocrates is basically identical to a standard Transformer Encoder. The layer normalization and the residual connections are implemented, respectively, with the classes `LayerNormalization` and `ResidualConnection`. Layer Normalization stabilizes and speeds up training by normalizing a layer's activations per token across its feature dimension, reducing sensitivity to weight initialization and activation scale, improving gradient flow, and enabling larger learning rates. Residual connections (skip connections) help train deep networks by letting layers learn a residual transformation on top of their input.

Before giving the tokens obtained from the tokenizer to the Encoder stack, we create an embedding of the tokens through `InputEmbeddings`. The embeddings are vectors representing each token in the input sequence, with a dimension defined by d_{model} . In this model, we set both the d_{model} and the maximum sequence length to **512**. This is done for computational limitation reasons, as this is the single most effective way to reduce memory usage and training time.

In this project, we tried some of the most famous attention mechanism, which we will show in the following sections. In each of the attention mechanism, we used **8** heads, as it allows a good compromise between capability of the model and computation time. One of the attention mechanisms is the one used in T5, hence **relative position encoding** with **Multi-head Attention**.

Multi-Head Attention with relative position encoding

Multi-Head Attention lets a model look at different parts of a sequence (or another sequence) in parallel, capturing complementary relations (syntax, semantics, locality, long-range links) in separate “heads.” Each head performs an attention operation on a lower-dimensional subspace; their results are concatenated and mixed. Given input representations $\mathbf{X} \in R^{n \times d_{\text{model}}}$, we form queries, keys, and values via learned linear maps:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V,$$

with $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in R^{d_{\text{model}} \times d_{\text{model}}}$. For h heads, the feature dimension is split so that each head uses $d_k = d_{\text{model}}/h$. In our implementation, h is set to 8. We must be sure that d_{model} is divisible by h .

After that, we calculate the scaled dot-product attention per Head as follows:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V},$$

where $\mathbf{M}$ is an additive mask (e.g., padding or causal) that disallows scores to specific positions by adding $-\infty$. In the encoder, the mask is relative to the padding tokens, so we mask only the positions relative to them. Heads are finally concatenated and projected:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1; \dots; \text{head}_h] \mathbf{W}^O, \quad \mathbf{W}^O \in R^{(h d_k) \times d_{\text{model}}}.$$

We use Multi-head attention because each head has its own projections ($\mathbf{W}_\ell^Q, \mathbf{W}_\ell^K, \mathbf{W}_\ell^V$) and can therefore learn distinct patterns (local n -grams, long-distance dependencies, coreference, etc.), increasing expressivity.

Self-attention needs **positional information**. Absolute encodings (sinusoidal or learned) add position vectors to embeddings. Relative encodings, instead, express *how far* (and in which direction) a key is from a query, often generalizing better across lengths and shifts. In the T5-style formulation, positional information is injected by *adding* a distance-dependent bias to the attention logits, without altering the content projections $\mathbf{Q}, \mathbf{K}$. For head ℓ , query index i , and key index j , the attention score becomes

$$\text{scores}_\ell(i, j) = \frac{\mathbf{Q}_\ell(i) \cdot \mathbf{K}_\ell(j)}{\sqrt{d_k}} + b_\ell(\text{bucket}(r)), \quad r = j - i,$$

where $b_\ell(\cdot)$ is an embedding over a *finite* set of distance buckets. The bias is then added *before* the softmax, thereby shaping relative preferences while remaining compatible with any padding/causal mask.

Directly learning a parameter per offset r is undesirable for long contexts. Instead, r is mapped to a small index via a *bucketing* function $\text{bucket} : Z \rightarrow \{0, \dots, B-1\}$. What we mainly do in relative position bias is:

1. **Direction split (bidirectional self-attention).** The B buckets are split evenly into “left” ($r < 0$) and “right” ($r > 0$) halves; $r = 0$ typically receives a dedicated small bucket. This lets the model distinguish “attend to the past” vs. “attend to the future.” In causal decoding, only $r \leq 0$ are used (future buckets are never queried).

2. **Exact vs. logarithmic regime.** Small absolute distances are given *fine-grained* buckets (e.g., $|r| = 1, 2, \dots$ up to a threshold m), while larger distances are quantized *logarithmically* into the remaining buckets:

$$\text{bucket}(|r|) = \begin{cases} |r|, & |r| < m, \\ m + \left\lfloor \frac{\log(|r|/m)}{\log(D/m)} (B/2 - m) \right\rfloor, & |r| \geq m, \end{cases}$$

where D is a chosen “max distance” at which the last bucket saturates. Intuitively, the model resolves nearby positions precisely and compresses far-away positions coarsely.

Combining direction and magnitude yields a single index in $\{0, \dots, B - 1\}$. Relative position bias introduces a small embedding table $E \in R^{B \times h}$, where B is the number of buckets and h is the number of heads. For each (i, j) , we compute $r = j - i$, map it to $u = \text{bucket}(r)$, and add the head-specific scalar $E[u, \ell]$ to the ℓ -th head logits at (i, j) . Broadcasting over the batch yields a bias tensor of shape (h, n_q, n_k) added to the raw scores in addition to the masks. In our implementation, the number of buckets is set to **32**, while the maximum distance D is set as **128**, like in the original T5 paper. The class in the code associated to Multi-Head Attention is called `MultiHeadAttentionBlock`

Multi-Head Latent attention with Decoupled RoPE

Multi-Head Latent Attention (MLA) was introduced in the DeepSeek-V2 architecture to cut the memory footprint of the key–value cache (KV cache, basically we store the K and V values for each new word generated instead of calculating again each time all the values) during autoregressive inference while preserving or improving model quality. The core idea is to store, for each token, a *single low-dimensional latent vector* instead of per-head keys and values, and to reconstruct per-head keys/values on the fly from this latent when computing attention. This turns the attention layer from KV–memory-bound into a more compute-heavy but cache-light operation—critical for efficient long-context generation.

$$\mathbf{c}_t^{\text{KV}} = \mathbf{W}^{\text{DKV}} \mathbf{h}_t, \quad (2.1)$$

$$\mathbf{k}_t^c = \mathbf{W}^{\text{UK}} \mathbf{c}_t^{\text{KV}}, \quad (2.2)$$

$$\mathbf{v}_t^c = \mathbf{W}^{\text{UV}} \mathbf{c}_t^{\text{KV}} \quad (2.3)$$

where $\mathbf{c}_t^{\text{KV}} \in R^{d_c}$ is the compressed latent vector for keys and values; $d_c (\ll d_h n_h)$ denotes the KV compression dimension; $\mathbf{W}^{\text{DKV}} \in R^{d_c \times d}$ is the down-projection matrix; and $\mathbf{W}^{\text{UK}}, \mathbf{W}^{\text{UV}} \in R^{d_h n_h \times d_c}$ are the up-projection matrices for keys and values, respectively. During inference, MLA only needs to cache $\mathbf{c}_t^{\text{KV}}$, so its KV cache has only $d_c l$ elements, where l denotes the number of layers. In addition, during inference, since $\mathbf{W}^{\text{UK}}$ can be absorbed into $\mathbf{W}^Q$ and $\mathbf{W}^{\text{UV}}$ can be absorbed into $\mathbf{W}^O$, we do not need to compute keys and values for attention.

To **inject positions efficiently**, MLA uses RoPE (Rotary Position Embeddings), a technique for encoding positional information. Unlike traditional positional embeddings that add a positional vector to the input embeddings, RoPE rotates the existing embedding vectors. The angle of rotation is determined by the token's absolute position in the sequence. The core mathematical insight is that the dot product of two rotated vectors is dependent on their relative angle. Given a query vector q_m at position m and a key vector k_n at position n , their embedding dimensions are paired. For each pair of dimensions i , a base frequency $\theta_i = 10000^{-2i/d}$ is defined, where d is the embedding dimension. The rotation for a token at position m is applied to each pair of features (x_j, x_{j+1}) in the embedding vector using a rotation matrix with an angle of $m\theta_i$. Specifically, the transformation is given by

$$\begin{pmatrix} x'_j \\ x'_{j+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_j \\ x_{j+1} \end{pmatrix}$$

After applying these rotations to both q_m and k_n , their dot product, which is fundamental to the self-attention mechanism, becomes a function of the relative position $m - n$. This property allows the model to inherently capture relative positional information and exhibits better generalization to sequence lengths not seen during training. The class in the for RoPE is called **RoPE**

A problem with RoPE is that it is incompatible with the compression method. To be specific, RoPE is position-sensitive for both keys and queries. If we apply RoPE for the keys, $\mathbf{k}_t^c$, $\mathbf{W}^{\text{UK}}$ will be coupled with a position-sensitive RoPE matrix and thus cannot be absorbed into $\mathbf{W}^Q$ during inference (matrix multiplication with RoPE does not obey a commutative law). As a result, we would need to recompute keys for all prefix tokens during inference, significantly hurting efficiency. To solve this problem, Decoupled RoPE was proposed. The intuition for Decoupled RoPE is to split the latent space for each head in two parts: content and rotary. In this way, we can apply RoPE on the rotary part, while preserving the content.

$$[\mathbf{q}_{t,1}^R, \mathbf{q}_{t,2}^R, \dots, \mathbf{q}_{t,n_h}^R] = \mathbf{q}_t^R = \text{RoPE}(\mathbf{W}^{\text{QR}} \mathbf{c}_t^Q), \quad (2.4)$$

$$\mathbf{k}_t^R = \text{RoPE}(\mathbf{W}^{\text{KR}} \mathbf{h}_t), \quad (2.5)$$

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^c; \mathbf{q}_{t,i}^R], \quad (2.6)$$

$$\mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^c; \mathbf{k}_t^R], \quad (2.7)$$

$$\mathbf{o}_{t,i} = \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^\top \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^c, \quad (2.8)$$

$$\mathbf{u}_t = \mathbf{W}^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] \quad (2.9)$$

where $\mathbf{W}^{\text{QR}} \in R^{d_h n_h \times d'_c}$ and $\mathbf{W}^{\text{KR}} \in R^{d_h^R \times d}$ are matrices to produce the decoupled queries and key, respectively; $\text{RoPE}(\cdot)$ applies rotary position embeddings; and $[\cdot; \cdot]$ denotes concatenation. In our implementation, the class relative to Multi-Head Latent Attention is called **MLADecoupledRoPe**, with the dimension of the latent is set to **128**, while the

dimension of the rotary part is set to 16. It's important to note that KV cache **is not implemented** in the code for two main reasons:

- We don't use KV cache in MHA, so to ensure a fair comparison between the two architectures, we removed any extra optimizations. This guarantees that any performance differences are due solely to the architecture itself.
- For a model of this reduced scale, operating on moderately long sequences, the KV cache would have offered a speed-up, but not a game-changing one. The study's focus was on validating the architecture's effectiveness, not on optimizing its deployment efficiency.

For this reason, the most coherent choice with the implementation of MHA was to not implement KV cache. In the code we can still find all the up-projection and down-projection matrices that are loyal to the original DeepSeek-V2 paper, with the possibility to easily implement KV cache.

Interleaved Attention Layers

Interleaved attention layers represent a sophisticated architectural design within Transformer-based models, where different types of attention mechanisms are strategically alternated within the network's structure. This approach moves beyond the use of a single, uniform attention mechanism across all layers. Instead, it combines distinct attention strategies to create a more efficient and powerful model. For instance, a common implementation involves interleaving *local attention*, which processes information within a smaller, defined window of the input sequence, with *global attention*, which captures long-range dependencies across the entire sequence. Another variation, particularly useful in tasks like feature matching, is to interleave *self-attention*, which allows a model to weigh the importance of different parts of a single input, with *cross-attention*, which enables the model to compare and relate different inputs.

The core principle behind this design is to enable the model to simultaneously process information at multiple scales and from various perspectives. By alternating between different attention types, the model can build a richer, more nuanced representation of the data. This strategic combination offers several key advantages. Firstly, it significantly enhances computational efficiency and reduces memory requirements, which is a critical factor for handling very long sequences. Secondly, by integrating both local and global information, interleaved attention layers can improve the model's accuracy and robustness, leading to more contextually relevant outputs. To implement Interleaved Attention Layers, we created a function `build_transformer_interleaved`, with the idea of alternating classic Multi-Head Attention layers and Multi-Head Latent Attention ones.

The `build_transformer` function takes as input a list of boolean values whose length is at most the number of layers in the encoder/decoder stacks. Inside the function we iterate over the input list: if the *i*-th value is `True`, the *i*-th layer of the encoder/decoder stack uses MLA as its attention mechanism; if it is `False`, that layer uses MHA as its attention mechanism. In our implementation, we decided for alternating MLA and MHA starting

with MHA. The idea is that we compress the input, and after we elaborate it better with MHA. Also other configurations can be tested, like first two layers of MHA and last two of MLA and viceversa, or alternating layers starting with MHA.

Feed Forward Network

The position-wise feed-forward network (FFN) in the encoder adds non-linearity and per-token feature transformation after attention. Concretely, each token vector x is processed independently by the same two-layer MLP, e.g.

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$$

which expands the dimensionality then projects it back. This gives the model greater representational capacity (learns higher-level features and combinations of attention outputs) and—together with residual connections and layer-norm—stabilizes and deepens the encoder.

In the code, there is a `FeedForwardBlock` class that implements the Feed Forward layer, with a hidden dimension of **2048** — four times larger than the latent space dimension, which, as mentioned earlier, is **512**. The activation function chosen is GeLU. The GeLU (Gaussian Error Linear Unit) activation was preferred over ReLU because it provides a smoother, probabilistic gating of neurons instead of a hard cutoff at zero. This allows small negative inputs to contribute slightly to the output, improving gradient flow and model stability.

2.2.2 Decoder

In the decoder, we still have the attention mechanism block, skip connections, normalization layers, and finally the Feed Forward layer. The only difference from the encoder is the addition of the cross-attention. Cross-attention is still an attention mechanism, but the key difference is that it takes K and V from the encoder's output, while Q depends on the decoder's input. In the case of self-attention, the input sequence to the decoder is masked, meaning that an additional mask is applied on top of the one used for padding tokens. This mask is called the **causal mask**, and it hides all tokens with positions $j > i$, i.e., “future” tokens.

2.2.3 Projection Layer and Softmax

The projection layer maps each decoder hidden state from the model space (d_{model}) to vocabulary logits (V). In other words, it converts features into scores for every token in the vocabulary so that we can compute a softmax and pick the **next token**. The projection layer is implemented in the code using the `ProjectionLayer` class, in this case without a softmax, because we use Cross-Entropy as loss, which already internally includes `log_softmax` and `NLLLoss`.

Chapter 3

Training of the model

3.1 Training a transformer

Given an input token sequence $x = (x_1, \dots, x_n)$ and a target sequence $y = (y_1, \dots, y_m)$, a Transformer learns $p(y|x)$ by factorizing

$$p(y|x) = \prod_{t=1}^m p(y_t | y_{<t}, x; \theta),$$

where θ are the model parameters. Training is supervised: we observe pairs (x, y) and minimize a token-level loss. The encoder learns a representation of the input tokens, creating K and V matrices that will be used from the decoder; the decoder generates y autoregressively, attending to both previously generated tokens and the output of the encoder. During training, the decoder receives the *gold* target shifted right, $y_{<t}$, and outputs logits per timestep; probabilities are the softmax of the logits. For each token we calculate the loss and we backpropagate the error, leading to model to predict more accurate texts w.r.t. to the target. In order to avoid to calculate the loss on padding tokens and on 'future tokens', we respectively use the encoder padding mask and the decoder **causal mask**, defined as a triangular matrix composed of 0 in its upper part, diagonal excluded. At test time the decoder runs autoregressively without teacher forcing. Search strategies include greedy decoding, beam search with length penalty, or sampling (top- k /nucleus). In this implementation we used greedy decoding, so we always choose the token with the highest probability after softmax. Decoding stops at an EOS token or a maximum length.

3.2 Training of NanoSocrates

NanoSocrates was trained to perform the 4 task mentioned in the previous chapters (Text2RDF, RDF2Text, RDF Completion 1 and RDF Completion 2) on a dataset composed of 1206 samples, based on 350 films in english language. So, the model is able of

understanding and translating from English to RDF and viceversa. Of these 1206 samples, the 90% is used for the training, while the remaining 10% is used for the evaluation of the model. The samples are randomly picked from the whole database, so basically the model is trained on **a mixture of the 4 tasks**. NanoSocrates, as we said when we were describing the architecture, is inspired to T5. Also for the training of the model, the samples are labeled with specific markers based on the task like T5, added to the vocabulary of the tokenizer. The markers are useful both to identify the task, and to calculate the appropriate metrics for the evaluation of the performances of the model. All training runs, including sanity checks, were performed on **Google Colab** using an **NVIDIA T4 GPU**.

3.2.1 Training setup

In order to train the model in the best way, we chose suitable scheduler, optimizer and scaler, with all the associated parameters. The parameters θ were initialized according to Xavier initialization.

- **Scheduler:** Training uses a two-phase learning-rate schedule: linear warmup from a small LR to the target LR, cosine decay from that target LR down to a tiny floor. These two phases are stitched together with a sequential scheduler so training seamlessly switches from warmup to decay at the chosen milestone (in our case, 5% of the total steps, considering the accumulation steps). The **reason** because we used this is that this is largely used for training Transformers. We also tried **OneCycle** scheduler with cosine decay with the idea of having a more aggressive training in the first epochs, but it worked worse. The cosine decay allows for a smoother training in the last epochs, where it is needed a bit more of "refinement", while the linear warm-up avoids initial instabilities;
- **Optimizer:** Parameters θ are updated with **AdamW**. AdamW is the gold standard of the optimizers, allowing to adapt the dimension of steps we take to the gradients we get
- **Scaler:** We used **GradScaler** as scaler. The main purpose of GradScaler in this project is because we worked with **autocast** in order to use low precision datatypes like **bfloat16**, mainly for faster calculations and to save memory due to hardware limitations. Since **bfloat16** and **float16** have a narrow dynamic range, small gradients can underflow to zero during backpropagation. So, GradScaler multiplies the loss by a scale factor before the backward, allowing to overcome this problem
- **Learning rate:** The learning rate start with a value of $4.0e-6$, that is $(1.0e-4)/25$, and has a maximum value of $1.0e-4$. This is done according to the division factor of the scheduler, that is 25. Common division factors values are in the range between 10 and 50.
- **Epochs:** The epochs are set to **80**, because the model is very small and the capacity of learning is quite limited considering that it has to learn four tasks. So, the model

stops improving around epoch 50/60. The last epochs are for refinement, according to the small learning rates provided by the scheduler. The epochs can be increased in case of a bigger model (so, increasing the number of heads, of encoder/decoder layers, hidden dimension, sequence length, etc.)

- **Batch size:** The batch size is set to **8**. With the accumulation steps, we could update the weights every n batches, apparently increasing the batch size. A little batch size helps for regularization (due to batch noise) and is good for long sequences (in this case, sequences of RDF triples and text). We have to adapt both the lr and the scheduler to the batch size, using smaller learning rates with tiny batches to avoid instabilities
- **Dropout:** Dropout is a regularization technique that "turns off" some of the activation with probability p , and scales the remaining activations by a factor $\frac{p}{1-p}$. We apply dropout in **Feed Forward**, on the attention scores, and on the embeddings. In the project, the dropout is set to **0.1**, avoiding too much regularization due to the small dimension of the model
- **Loss:** The loss is the **CrossEntropyLoss**, which expects an unnormalized score for each class. It is useful when training a classification problem with C classes.
- **Label Smoothing:** In standard cross-entropy training, the target for each position is one-hot: all the probability mass is placed on the correct class and zero on all others. Label smoothing replaces that vector with a "soft" version, distributing a small portion of probability mass across the incorrect classes as well. In the code, label smoothing is set to **1.0**

3.2.2 Input sequences

For each sample created with the `database_creator.py` script, we obtain one JSON line containing two fields: `"input"` and `"target"`. First, we tokenize both the input and the target using the tokenizer previously trained on the curated corpus. This produces three sequences used during model training: the encoder input, the decoder input (used for teacher forcing), and the target sequence. Any tokenized sequence longer than the maximum limit (512 tokens in our case) is truncated. We then distinguish three cases:

- **Encoder input:** The encoder receives the tokenized sequence corresponding to the `"input"` field of the sample. We prepend the `[BOS]` token and append the `[EOS]` token to mark the beginning and end of the sequence.
- **Decoder input:** The decoder receives the tokenized sequence corresponding to the `"target"` field of the sample, but only with the `[BOS]` token at the beginning. This sequence is used during teacher forcing in training: the model is fed the gold previous tokens (properly masked), regardless of what it predicted at the previous step.

- **Target:** The target sequence is used to compare the predicted tokens and compute the loss. It is identical to the decoder input, but with the [EOS] token appended at the end, and without the [BOS] token.

In the decoder input, task-related tags are also included. These tags are preserved to indicate which task the model must perform—similar to the input format used by T5. Additionally, the tags are used in the evaluation code to determine which metrics should be applied to assess the model’s predictions for each example. If a tokenized sequence is shorter than the maximum length, [PAD] tokens are appended until that length is reached to ensure uniform sequence shapes. During training, these padding tokens are masked in both self-attention and cross-attention computations so that they do not affect the attention scores.

3.2.3 Sanity check on the model

Before training the model on the entire dataset, we perform a **sanity check** by training the model repeatedly on a single batch of examples. This allows us to verify whether the loss decreases toward zero and whether the model’s predictions become nearly perfect. The sanity check provides insight into the model’s ability to learn, and therefore into whether the architecture has been implemented correctly. To allow the model to completely overfit the input data, we disable both label smoothing and dropout in every part of the model, set the weight decay of AdamW to zero, and train for 100 epochs. When using each type of attention, the model correctly learns to predict the target, achieving a loss very close to zero and generating outputs identical to the ground truth. The sanity check was essential to detect an **issue with the attention masks**: in the first implementation, they were incorrect, as the [BOS] and [EOS] tokens were still included in the decoder input. This prevented the model from learning properly, causing almost entirely incorrect predictions. In the code, the function to perform the sanity check is called `sanity_check`.

Source	Target	Predicted
[BOS]<SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:director<OBJ>dbr:James_Cullen_Bressack<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:producer<OBJ>dbr:James_Cullen_Bressack<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:producer<OBJ>dbr:Timothy_Woodward_Jr.<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Bill_Cobbs<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:DMX<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Johnny_Messner_(actor)<EOT><CONTINUE>	<SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Steven_Seagal<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Zack_Ward<EOT>	<SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Steven_Seagal<EOT><SOT><SUBJ>dbr:Beyond_the_Law_(2019_film)<PRED>dbr:starring<OBJ>dbr:Zack_Ward<EOT>

Table 3.1: Sanity check on a single batch — example 0. Source, target and predicted sequences.

3.2.4 Setting of the parameters

All the parameters we discussed before, can be set from the `config` in the model, included the directories to save the weights of the model and the results of the experiments. The weights are saved every 20 epochs (so, at epoch 19, 39, 59, and so on), while the results are saved every 5 epochs. The config is basically a vocabulary, used to control the parameters of the model. Here we can also set if to use Multi-Head Attention standard or Latent (`'use_mla'`), or if to use Interleaved Attention Layers, passing in the config the attention pattern (`interleave_pattern`). We can also control if to perform or not the sanity check by using the parameter `'sanity_one_batch'`

Chapter 4

Evaluation of the model

To evaluate the model's performance, we use different metrics depending on the task, each chosen to properly assess the model's learning capability. To manage computational costs, a two-tiered validation strategy was employed. A comprehensive ('FULL') validation was conducted every 20 epochs to obtain reliable performance metrics. In the intervening epochs, a streamlined ('FAST') validation was run. This faster evaluation was achieved by setting limits on the number of batches to evaluate, on tasks involving triples, on text generation length, and on the predictions for the `<MASK>` token. Furthermore, the calculation of the computationally intensive METEOR score was skipped. Consequently, these intermediate results should be considered indicative, as their primary purpose was to provide a quick directional check on the model's training progress. A full evaluation can be run on any epoch by setting the `eval_profile` parameter to FULL. In the following sections, we will evaluate the performance task by task, also examining representative examples of the model's generations and discussing their strengths and weaknesses.

4.1 Learning Curves



Figure 4.1: Learning curves for training and validation

The above figure illustrates the model’s training and validation loss curves over 80 epochs. The training loss (orange line) shows a consistent and smooth decrease, indicating that the model successfully learned the patterns within the training data, eventually plateauing as it reached its learning capacity. The validation loss (blue line) initially decreases in tandem with the training loss, demonstrating effective generalization to unseen data. However, a key result is observed around epoch 50, where the validation loss stops improving and flattens, while the training loss continues to decline. This divergence is a classic sign of overfitting, suggesting that after this point, the model began to memorize the training set rather than learning generalizable features. This confirms that the optimal model performance on new data is achieved around epochs 40-50, beyond which further training does not yield better generalization. The results are available in the `learning_curves.csv` file.

4.2 Text2RDF

To evaluate Text2RDF, we used Recall, Precision, and F1-score. We do not compute the metrics on each individual sample (i.e., by calculating precision, recall, and F1 for each sample and then averaging them), but rather by accumulating the true positives (TP), the number of predicted tokens, and the number of target tokens across all samples, and then computing the metrics at the end.

- **Precision** is defined as $\frac{TP}{\text{Number of predicted tokens}}$. True Positives are the tokens that model correctly predicts, so that correspond to the target ones, while False Positives are tokens that model predicts, but that are not in ground truth. So, since $\text{Number of predicted tokens} = TP + FP$, we can consider the precision as defined below.
- **Recall** is defined as $\frac{TP}{\text{Number of tokens of the target}}$. The False Negatives are the tokens of the target sequence that are not predicted by the model, so we can consider $\text{Number of tokens of the target} = TP + FN$.
- **F1 Score** is defined as $\frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$

The results are as follows on the last validation after 80 epochs for each attention mechanism:

	Precision	Recall	F1-score
MHA	0.6032	0.5206	0.5589
MLA	0.5564	0.5457	0.5510
Interleaved	0.6072	0.5569	0.5809

Table 4.1: Results for Text2RDF

We consider the model’s results to be solid and consistent with the scale of the model and the dataset, particularly given the available computational constraints. The primary challenge for the model in this task—which is inevitably mirrored in its counterpart, the

RDF2Text task—lies in genuinely grasping the correspondence between an RDF triple and its textual meaning. This difficulty arises because the information expressed in Wikipedia abstracts often does not perfectly align with the data contained in the RDF triples, either by adding extra context or by omitting it. As a result, the model is exposed to significant ‘noise’ in the input data, which hinders its ability to accurately capture the underlying semantics. However, the model understands the structure of an RDF triple, being able to generate triples with the correct tags in the right places, also using the correct resources, as also shown by the Precision and Recall values.

4.3 RDF2Text

To evaluate the model’s performance on the RDF2Text task, we use the ROUGE-L, BLEU-4, and METEOR metrics.

- **BLEU-4 (Bilingual Evaluation Understudy):** A precision-focused metric that measures the overlap of word n-grams (up to 4-grams) between the generated and reference texts. It is a standard metric for translation and text generation quality. It ranges between 0 and 1
- **ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation):** A recall-focused metric that measures the longest common subsequence (LCS) of words between the generated and reference texts. It excels at capturing sentence-level structural similarity. It also ranges between 0 and 1
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering):** An advanced metric that evaluates text by aligning words based on exact matches, stems, and synonyms. It balances precision and recall and is designed to better correlate with human judgment. Ranges between 0 and 1.

The results are as follows on the last validation after 80 epochs for each attention mechanism:

	BLEU-4	ROUGE-L	METEOR
MHA	0.0324	0.2254	0.1715
MLA	0.0346	0.2160	0.1816
Interleaved	0.0255	0.2056	0.1775

Table 4.2: Results for RDF2Text

As already said before, the main problem in text generation, is that, in the majority of the cases, we don’t have a perfect match between informations contained in the RDF Triples and the abstract from Wikipedia. So, the model struggles in having good results in this kind of task.

4.4 RDF Completion 1 - Spanned Masking

The RDF Completion 1 task is evaluated using two primary metrics:

- **Token-Level Accuracy:** This is calculated on individual tokens and is formally equivalent to the Recall metric defined previously (i.e., the sum of True Positives divided by the total length of the reference sequence across all instances).
- **Masked Span Accuracy:** This metric assesses the model's ability to predict the entire sequence of target tokens corresponding to the `<MASK>` token. For its calculation, we use a Counter data structure. This implementation introduces a tolerance for the generation order of the tokens. However, it remains strict in that both the predicted tokens and their frequencies must be identical to those in the ground truth.

The results are as follows on the last validation after 80 epochs for each attention mechanism:

	Token-Level Accuracy	Masked Span Accuracy
MHA	0.6332	0.2121
MLA	0.6271	0.2121
Interleaved	0.6359	0.2424

Table 4.3: Results for RDF Completion 1 - Spanned Masking

Looking at the results and the sequences predicted by the model, it is generally effective at predicting the correct role, but it fails to link the triple to the right resource or property. We still believe this is a great outcome, which can be further improved by adding more training data and increasing the model's size. Following, an example correctly generated, and one that is only predicting correctly the role:

Source	Target	Predicted
<code><SOT><SUBJ>dbr:Berkeley_Square_(1933_film)<MASK><OBJ>dbr:Heather_Angel_(actress)<EOT></code>	<code><PRED>dbo:starring</code>	<code><PRED>dbo:starring</code>
<code><SOT><MASK><PRED>dbo:starring<OBJ>dbr:Lorne_Michael<EOT></code>	<code><SUBJ>dbr:Love,_Gilda</code>	<code><SUBJ>dbr:Let's_All_Hate_Toronto</code>

Table 4.4: Examples for RDF Completion 1 – Spanned Masking (example)

4.5 RDF Completion 2

For the RDF Completion 2 task, we again use Precision, Recall, and F1-score, defined identically to those in the Text2RDF task. The results are as follows on the last validation after 80 epochs for each attention mechanism:

	Precision	Recall	F1-score
MHA	0.5398	0.8008	0.6649
MLA	0.5634	0.8669	0.6830
Interleaved	0.5285	0.6463	0.5815

Table 4.5: Results for Text2RDF

The model demonstrates an ability to comprehend the structure of the triples and the correct placement of resources and predicates. However, it still faces challenges in correctly continuing the generation process for longer sequences. Despite this, we believe the results are strong, especially when considering the scale of the model and the dataset. The main thing to observe is the very strong Recall values that both MLA and MHA show. This means that are highly effective at generating the complete set of triples required by the input text, successfully capturing and outputting most of the ground-truth information.



Chapter 5

Conclusions and future Developments

This project successfully achieved its **primary objective**: the creation of a small foundational model, "NanoSocrates," built from scratch with a T5-like encoder-decoder architecture. The model was designed to bridge the gap between unstructured natural language and structured RDF data, addressing four distinct tasks: Text2RDF, RDF2Text, and two variants of RDF completion. Moreover, the model tested some variants of the attention mechanism, such as Latent Attention and Interleaved Attention Layers. The development process encompassed the entire pipeline, from the creation of a custom, reproducible dataset derived from DBpedia and Wikipedia to the training of a bespoke BPE tokenizer.

The evaluation across the four tasks demonstrates that NanoSocrates, despite its limited scale, successfully learned the fundamental syntax and structure of RDF triples. In tasks like Text2RDF and RDF Completion 2, the model showed a clear ability to generate correctly formatted triples, placing resources and predicates appropriately. This indicates a solid grasp of the structured data format.

However, the **primary challenge** encountered was the inherent semantic mismatch between the concise, factual nature of RDF triples and the richer, more contextual information present in Wikipedia abstracts. This "noise" in the training data was the main obstacle to achieving higher performance, particularly in the RDF2Text task. Specific limitations were observed where the model could, for instance, predict the correct role in a masked triple but failed to identify the precise resource or property, or struggled to maintain coherence over longer sequences of generated triples.

Considering the constraints of the model's size and the dataset, the results are very promising. The project serves as a strong proof-of-concept for the viability of small, specialized foundational models. For future work, two clear paths for improvement emerge directly from our findings:

- **Data Scaling:** Expanding the training dataset with more examples would expose the model to a wider variety of linguistic and structural patterns, likely improving its ability to generalize and handle the semantic gap more effectively.

- **Model Scaling:** Increasing the model's architectural size—for instance, by adding more layers or attention heads—would enhance its capacity to learn more complex relationships within the data.

By pursuing these directions, the capabilities of NanoSocrates can be significantly enhanced, further advancing its ability to seamlessly translate between the structured world of RDF and the fluid nuances of natural language.